

GET 324: Computer Engineering Laboratory Course



Technical Project Report: Tomato Leaf Disease Classification

Group ID: C02

Task: Binary Classification (Healthy Tomato Leaves vs. Tomato Mosaic Virus)

Date: July 29, 2026



Group Members & Contributions

- ***Role*:** Technical Report Writing, Methodology Analysis, Documentation Lead, and Git Repository Maintenance.
 - ***Role*:** Dataset Sourcing, Verification, Split-Folder Operations, and Data Pipeline Optimization.
 - ***Role*:** Model Verification, Test Dataset Evaluation, Confusion Matrix Testing, and Quality Assurance.
 - ***Role*:** System Integration Testing, Local Environment Validation, and Performance Review Support.
-

Abstract

This report presents the design, implementation, and deployment of a deep-learning-based computer vision system to automate the identification of the **Tomato Mosaic Virus**—a highly contagious plant pathogen. Utilizing the **PlantVillage Dataset**, we constructed a complete machine learning workflow. We experimented with two distinct architectures: a custom three-block Convolutional Neural Network (CNN) built from scratch, and a transfer learning approach leveraging a pre-trained **MobileNetV3Small** backbone. The transfer learning model achieved a validation accuracy of **99.66%** and generalized to **100% accuracy** on unseen test data. The final system was successfully integrated into an interactive Streamlit web dashboard and deployed to the Streamlit Community Cloud for real-time agricultural diagnosis.

1. Introduction & Objectives

Early and accurate detection of crop diseases is essential for securing food supply chains and minimizing economic losses in agriculture. The **Tomato Mosaic Virus (ToMV)** causes leaves to develop mottling, chlorosis, and dwarfing, drastically reducing fruit yield. Traditional diagnosis relies on visual inspection by agricultural experts, which is slow, error-prone, and scaling-limited.

This project aims to:

- Build a deep learning pipeline to classify tomato leaf images as **Healthy** or infected with **Tomato Mosaic Virus**.
 - Implement and compare a **Custom CNN** against a **MobileNetV3** transfer learning model.
 - Develop a production-ready **Streamlit** app enabling users to upload images and receive predictions with confidence scores.
 - Host the application on the cloud to make it accessible to farmers and field researchers.
-

2. Environment Verification & Data Pipeline

2.1. Libraries & Random Seed Initialization

To guarantee reproducibility across different runtime environments, strict environment parameters were set:

- TensorFlow version `2.15+` was verified.
- The random seed `SEED = 42` was initialized across `random`, `numpy`, and `tensorflow`.
- Deterministic operations were enforced using `os.environ['TF_DETERMINISTIC_OPS'] = '1'`.
- GPU availability was verified using `tf.config.list_physical_devices('GPU')`.

2.2. Dataset Source & Class Filtering

The dataset was obtained from the **PlantVillage Dataset**, a publicly available collection of healthy and diseased crop leaf images. For this binary classification task, we filtered the following subsets:

1. `Tomato_healthy` (Healthy class)
2. `Tomato__Tomato_mosaic_virus` (Diseased class)

The selected images were copied into a filtered local working directory using python's `shutil` utility.

2.3. Training, Validation, and Test Splits

The filtered dataset was partitioned into train, validation, and test subsets using `split-folders` in a **70% / 15% / 15%** ratio.

- **Train Set:** Used to optimize the model parameters (weights and biases).
- **Validation Set:** Used for parameter tuning and monitoring overfitting during training.
- **Test Set:** Completely held out until the final evaluation to report unbiased performance.

The dataset folders were verified programmatically using directory globbing:

```
train_dir = BASE_DIR / 'train'

val_dir = BASE_DIR / 'val'

test_dir = BASE_DIR / 'test'
```

2.4. Loading & Pipeline Optimization

The data was loaded into memory using Keras' `image_dataset_from_directory` utility, resizing images to a standard shape of `224 × 224` pixels with a batch size of `32`. Keras sorted the classes alphabetically:

- **Index 0:** `Tomato__Tomato_mosaic_virus` (Diseased)
- **Index 1:** `Tomato_healthy` (Healthy)

The input data pipeline was optimized using TensorFlow's `tf.data` API to prevent CPU-GPU starvation bottlenecks:

```
AUTOTUNE = tf.data.AUTOTUNE

train_dataset = train_dataset.cache().shuffle(1000).prefetch(buf

valid_dataset = val_dataset.cache().prefetch(buffer_size=AUTOTUN

test_dataset = test_dataset.cache().prefetch(buffer_size=AUTOTUN
```

- `.cache()` retains images in RAM after the first epoch, eliminating slow disk read times.
- `.shuffle(1000)` randomizes batch contents in memory to improve regularization.
- `.prefetch(AUTOTUNE)` overlaps data preprocessing and model execution, keeping the GPU constantly busy.

3. Data Augmentation

To mitigate overfitting due to small sample sizes and variance in real-world agricultural imagery (e.g., shadows, camera angles), we constructed a sequential data augmentation block:

```
data_augmentation = tf.keras.Sequential([  
    tf.keras.layers.RandomFlip('horizontal'),  
    tf.keras.layers.RandomRotation(0.2),  
    tf.keras.layers.RandomZoom(0.2),  
], name='data_augmentation')
```

During training, this block dynamically applies random horizontal reflections, rotations up to 72° , and zooming up to 20%. In inference (validation and testing), this layer is automatically bypassed to maintain stable predictions.

4. Model Architectures

4.1. Custom Three-Block CNN (From Scratch)

A standard Convolutional Neural Network (CNN) was designed to act as a baseline. The architecture doubles convolutional filters at each block to capture low-level texture details in early layers, and abstract shape representations in deeper layers.